

COMP4128 Programming Challenge Portfolio

Apply the C++ programming language to solve programming problems

When solving Bitmask DP problems like E. Keyboard Purchase (Problem Set 4), I applied C++ language features very often to optimize my solution. For example, I took advantage of the built-in bit manipulation function `__builtin_ctz()` to efficiently extract the index of the new added character in the bitmask, which is much more efficient and safer than writing a manual loop. Additionally, I proficiently used `std::vector` from the C++ to manage the 2D frequency array and the DP state table, ensuring safe memory allocation and boundary management. These language level applications made the originally complex binary state transition logic very clear and easy to implement and understand. This example fully demonstrated I meet the CLO applying the C++ programming language to solve programming problems.

Adapt templated data structures and algorithms to meet the requirements of complex problems

When tackling E. Points (Problem Set 3), I fully understood how to modify classic data structures based on specific requirements of a complex question. The coordinate range given by the problem is up to 10^9 , making it impossible to build a standard segment tree directly with the current space limit. I first applied compression of the coordinates, combining `std::unique` and `std::sort` to map the sparse coordinate system to a dense interval from 1 to N . More importantly, I adapted the node structure of a standard segment tree by embedding a `std::set` within its leaf nodes. This modification allowed me to dynamically maintain the set of maximum Y-coordinates for points sharing the same X-coordinate, thereby efficiently handling dynamic point insertions/deletions and 2D partial order queries in a log time. This is a strong example for me to adapt templated data structures and algorithms to meet the requirements of complex problems.

Design and implement algorithmic solutions to complex problems, particularly under short time constraints

In Trapping Tourists (Contest 3), the problem required assigning positive integer prices to edges in a directed graph such that every journey belongs to at least one minimum cost path from Aachen to Zwolle. I analyzed the mathematical constraints of shortest paths and successfully deduced the core condition: because all weights must be strictly positive, traversing any cycle would strictly increase the total path cost. Therefore, the absolute prerequisite for any valid assignment is that the graph of viable paths must be a DAG meaning it cannot contain any SCC. By immediately reducing the complex pricing problem into an SCC detection task which can be solved efficiently using Tarjan's or Kosaraju's algorithm, I designed a solution that bypassed brute force evaluation and efficiently solved the problem within the contest's short time limit.

Apply algorithm design principles to evaluate programs in terms of correctness and running time

In E. Ancient Berland Roads (Problem Set 2), the problem requires tracking the maximum population of connected components over multiple queries involving road closures and population updates. A naive brute force simulation by rebuilding the graph and running a DFS for each query would yield a time complexity of $O(q \cdot (n + m))$, resulting in roughly 10^{11} operations that would severely exceed the time limit. To evaluate alternative correctness and efficiency principles, I recognized that standard DSU excels at merging components rather than splitting them apart as required by road deletion queries. By redesigning the algorithm to process queries offline from back to front, I transformed road deletions into edge merging operations and managed population changes dynamically. This design choice successfully evaluated and optimized the time complexity to fit within the constraints while ensuring absolute logical correctness.